

Roll No: BL.SC.U4AIE24105
Name: Ankur De
Lab 1

Q1

Generate the probability distribution for all the possible states of the coin, and the sum of the two dice being thrown.

Approach

I am using simulation to generate the distribution

Code

```
package com.ankur.Lab_1.Q1;

import java.util.Arrays;

/**
 * Assume a fair coin, and a simple pair of fair dice are being thrown.
 * a) Generate the probability distribution for all the possible states of the coin, and the sum of
 the two
 * dice being thrown.
 */

class CoinDice {
    private final int runs;

    CoinDice(int runs) {
        this.runs = runs;
    }

    private static int random(final int lowerBound, final int higherBound) {
        return (int) (Math.random() * ((higherBound - lowerBound) + 1)) + lowerBound;
    }

    private static int genDice() {
        return random(1, 6);
    }

    private static int genCoin() {
        return random(0, 1);
    }

    String run() {
        final int coinFaces = 2;
```

```

        int[][] distribution = new int[coinFaces][13];

        for (int i = 0; i < this.runs; i++) {
            int dice1 = genDice();
            int dice2 = genDice();
            int sum = dice1 + dice2;
            int coin = genCoin();
            distribution[coin][sum]++;
        }

        StringBuilder sb = new StringBuilder();

        for (int[] eachCoin : distribution) {
            sb.append(Arrays.toString(eachCoin)).append(",\n");
        }

        return sb.toString();
    }
}

public class Q1 {
    static void main() {
        CoinDice coin = new CoinDice(1000000);
        IO.println(coin.run());
    }
}

```

Terminal Output:

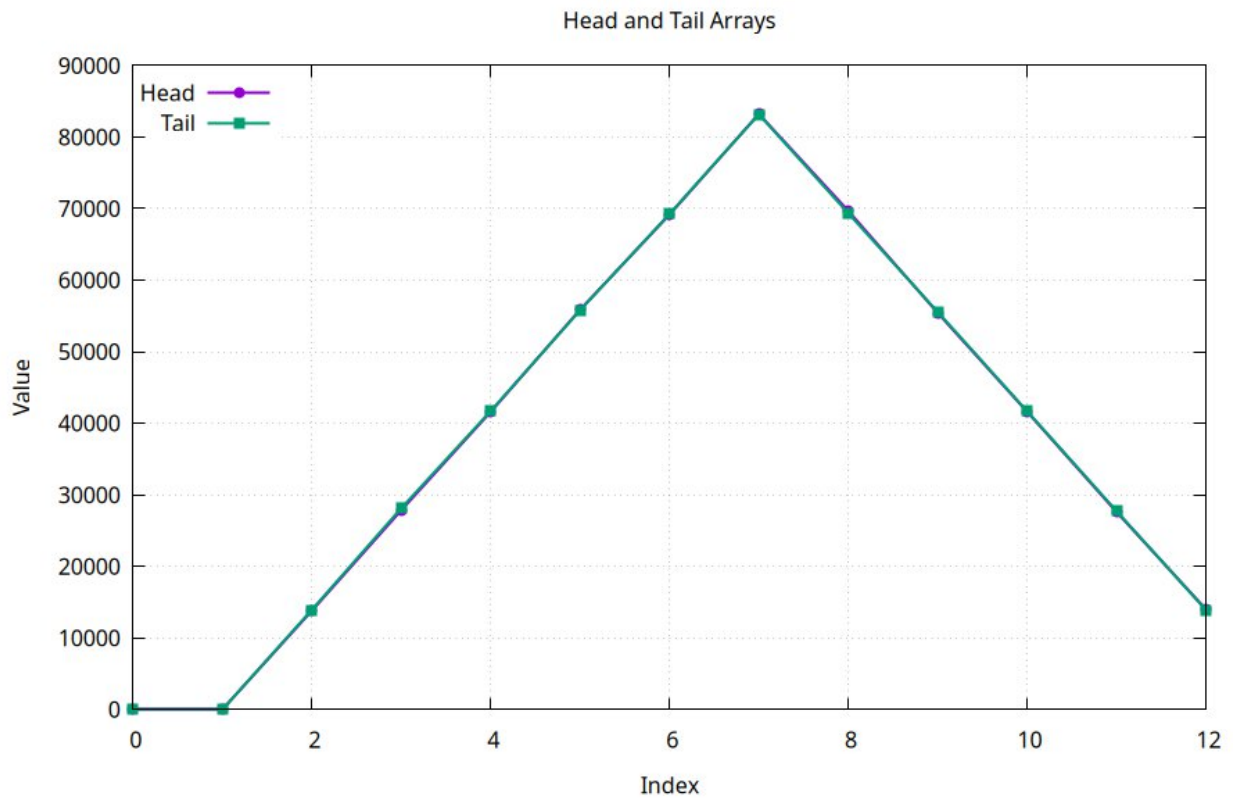
```

/usr/java/jdk-25-oracle-x64/bin/java -javaagent:/snap/intellij-idea-ultimate/798/lib/idea_rt.jar=39767 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath /media/ankur-de/Data/Ankur/Study/Semester 5/Probablistic Reasoning/Lab/Probablistic Reasoning/target/classes com.ankur.Lab_1.Q1.Q1
[0, 0, 13759, 27906, 41626, 55211, 69593, 83585, 69079, 55529, 41811, 27835, 13829],
[0, 0, 13773, 27685, 41603, 55881, 69572, 83132, 69534, 55708, 41804, 27797, 13748],

```

Process finished with exit code 0

Plot output



Q2

Assume that a person stands at position 0 at time 0. He throws a fair sided die. If he gets an even number, he will take that many steps forward. In other words, if he gets 2, he takes two steps forward. If he gets 4, he takes four steps forward, and if he gets 6, he takes six steps forward. On the other hand, if he gets an odd number, he takes the corresponding number of steps backwards. In other words, 1 results in one step back, 3 in 3 steps back, and 5 in 5 steps back. Generate the probability distribution for all the states after 10 dice throws.

After 10 dice throws, what is the probability that the person is at positions

- i) 0
- ii) 5
- iii) 10

Approach

I am using simulation to generate the distribution

Code

```
package com.ankur.Lab_1.Q2;

import java.util.Arrays;

/**
 * Assume that a person stands at position 0 at time 0. He throws a fair sided die. If he gets an
 * even
 * number, he will take that many steps forward. In other words, if he gets 2, he takes two steps
 * forward.
 * If he gets 4, he takes four steps forward, and if he gets 6, he takes six steps forward. On the
 * other hand,
 * if he gets an odd number, he takes the corresponding number of steps backwards. In other
 * words, 1
 * results in one step back, 3 in 3 steps back, and 5 in 5 steps back. Generate the probability
 * distribution
 * for all the states after 10 dice throws.
 */

class WalkingMan {
    private final int runs, noSteps;
    double[] arr;
    int minVal, maxVal;

    WalkingMan(int runs, int steps) {
        this.runs = runs;
        this.noSteps = steps;
    }

    private static int random(final int lowerBound, final int higherBound) {
        return (int) (Math.random() * ((higherBound - lowerBound) + 1)) + lowerBound;
    }

    private static int genDice() {
        return random(1, 6);
    }

    public void run() {
        this.minVal = -5 * noSteps;
        this.maxVal = 6 * noSteps;
        this.arr = new double[Math.abs(minVal) + maxVal + 1];

        for (int i = 0; i < this.runs; i++) {
            int pos = 0;
```

```

    for (int j = 0; j < this.noSteps; j++) {
        int step = genDice();

        // Logic for even odd detection
        // Checking for odd by logic number & 1 == 1 means odd
        if ((step & 1) == 1) {
            // Odd
            pos -= step;
        } else {
            pos += step;
        }
    }
    arr[pos + Math.abs(minVal)]++;
}

arr = Arrays.stream(arr)
    .parallel()
    .map(element -> element / runs)
    .toArray();
}

public double prob(int pos) {
    return arr[Math.abs(minVal)-pos];
}

@Override
public String toString() {
    return Arrays.toString(this.arr);
}
}

public class Q2 {
    static void main() {
        WalkingMan walkingMan = new WalkingMan(60_000_000, 10);
        walkingMan.run();
        System.out.println(walkingMan);
        System.out.printf("positions \n0: %f, \n5: %f,\n10: %f,", walkingMan.prob(0),
        walkingMan.prob(5), walkingMan.prob(10));
    }
}

```

Terminal output

```

/usr/java/jdk-25-oracle-x64/bin/java -javaagent:/snap/intellij-idea-ultimate/798/lib/idea_rt.jar=40261 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath /media/ankur-de/Data/Ankur/Study/Semester 5/Probabilistic Reasoning/Lab/Probabilistic Reasoning/target/classes com.ankur.Lab_1.Q2.Q2
[0.0, 0.0, 1.6666666666666668E-7, 0.0, 1.05E-6, 0.0, 3.8E-6, 8.333333333333334E-8,
9.716666666666666E-6, 1.65E-6, 2.445E-5, 9.633333333333333E-6, 4.785E-5, 3.645E-5,

```

```
8.185E-5, 1.0061666666666667E-4, 1.197E-4, 2.4076666666666667E-4,
1.7581666666666667E-4, 4.719333333333333E-4, 3.0621666666666667E-4,
7.860666666666666E-4, 5.974E-4, 0.0011377, 0.00118745, 0.0014839, 0.0021921, 0.001893,
0.003577, 0.0026025166666666668, 0.005092016666666667, 0.0040020333333333335,
0.006407533333333333, 0.006434733333333333, 0.0074062833333333334,
0.0098985166666666666, 0.00836195, 0.01371045, 0.010057, 0.016881266666666665,
0.0134385666666666667, 0.018653816666666666, 0.01857835, 0.0191525,
0.024623783333333333, 0.019466333333333332, 0.029644466666666668, 0.0212814,
0.031972833333333333, 0.02541345, 0.031184633333333333, 0.0311134,
0.028526033333333332, 0.03610445, 0.026382566666666666, 0.03815966666666667,
0.026367616666666666, 0.03609305, 0.028563033333333335, 0.031101866666666665,
0.031178416666666667, 0.025447666666666667, 0.031989633333333334, 0.0212824,
0.029641, 0.019476633333333333, 0.024608333333333333, 0.019126733333333333,
0.018550983333333333, 0.018713333333333332, 0.013428516666666666,
0.0168956166666666665, 0.0100689, 0.01371865, 0.008352716666666666,
0.0098658166666666668, 0.00740365, 0.0064555, 0.00641215, 0.0040062833333333333,
0.0050923166666666667, 0.002604, 0.0035808833333333333, 0.00190155,
0.0022055666666666667, 0.0014851833333333333, 0.00119155, 0.0011338333333333333,
5.953666666666667E-4, 7.937666666666666E-4, 3.045333333333333E-4,
4.685833333333333E-4, 1.819833333333333E-4, 2.396166666666667E-4,
1.218833333333333E-4, 1.0303333333333334E-4, 7.863333333333334E-5,
3.493333333333333E-5, 4.691666666666666E-5, 9.316666666666667E-6,
2.3833333333333334E-5, 1.833333333333333E-6, 1.048333333333333E-5,
1.3333333333333334E-7, 3.666666666666666E-6, 0.0, 9.833333333333334E-7, 0.0,
1.833333333333333E-7, 0.0, 1.666666666666667E-8]
positions
0: 0.031185,
5: 0.019466,
10: 0.013439,
Process finished with exit code 0
```

Plot Output:

